



# 8DM60: Multimodal AI in Biomedicine

## Lecture 6: Agentic AI

Mitko Veta

Department of Biomedical Engineering

# From next-token prediction to useful responses

Given earlier tokens  $x_{<t}$ , the model predicts  $p_{\theta}(x_t | x_{<t})$ .



- **Pretraining:** learn to predict text.
- **Instruction tuning:** train on requests and desired responses.
- **Inference:** generate tokens with fixed weights.

# What is a reasoning LLM?

A model trained to use intermediate steps to solve multi-step problems.

- **Training:** learn from worked solutions or rewards for successful answers.
- **Inference:** generate steps that help plan, check and revise a solution.

More reasoning spends computation before answering. Weights stay fixed during use.

**Example: DeepSeek-R1.**

## How are the “thoughts” generated?

Question: 3 trays, 4 samples each; 2 fail. How many remain?  
<think> 3 times 4 is 12. Subtract 2 to get 10. </think>  
<answer> 10 samples remain. </answer>

$$p_{\theta}(\text{next token} \mid \text{question, preceding reasoning tokens})$$

- The same generation loop produces reasoning and answer tokens.
- Earlier steps become context for later steps.
- These explanations may omit factors that influenced the answer.

Source: [R1-Zero training template](#); illustrative text, model-dependent markers.

# The augmented LLM: retrieval, tools and memory

- **Retrieval:** add relevant information to the context.
- **Tools:** request external operations with defined inputs and outputs.
- **Memory:** retain information for later use.

A tool can search, calculate or run an imaging model.

Its description tells the LLM what it does and which inputs it needs.

Source: [Building effective agents \(2024\)](#).

# Who executes a tool request?

**LLM: generates a request**

```
{"tool": "calculate", "expression": "12 - 2"}
```

**Harness: runs the calculator and adds its result to context**

Observation: 10

The next prediction uses this result alongside the question and earlier text.

The observation's label identifies it as **tool output**.

# Tool calling is a capability; ReAct is a strategy

## Tool calling

Request an external operation and receive its result.

## ReAct

Generate a “thought”, choose an action, observe the result, then reason again.

Tool calling can appear in a fixed workflow or an agent.

ReAct uses it within a **reasoning-action loop**.

Source: [Yao et al., ReAct \(2023\)](#).

## ReAct: generated text meets the environment

| Step        | Source                | Effect                       |
|-------------|-----------------------|------------------------------|
| "Thought"   | LLM-generated text    | Tracks or revises the plan   |
| Action      | LLM-generated request | Harness executes the command |
| Observation | Tool or environment   | Harness adds it to context   |

"Thought" → Action → Observation → "Thought" → ...

ReAct combines an LLM with an external execution loop.

Source: [Yao et al., ReAct \(2023\), Sections 2-3.](#)

# Each observation changes the next model context

**Question: How many slides in study S pass quality control?**

|             |                                             |
|-------------|---------------------------------------------|
| "Thought"   | I need the study's quality-control results. |
| Action      | <code>read_study("S")</code>                |
| Observation | 12 slides; 2 failed quality control.        |
| "Thought"   | Subtract the failed slides from the total.  |
| Action      | <code>calculate("12 - 2")</code>            |
| Observation | 10                                          |
| Answer      | 10 slides pass quality control.             |

$$p_{\theta}(x_t \mid q, T_1, A_1, O_1, T_2, A_2, O_2)$$

*T*: "thought"; *A*: action; *O*: observation. Illustrative study and trajectory.

Source: Yao et al., ReAct (2023).

# Few-shot examples show the model what to do

**Before the new question, include worked examples:**

Question + “thought” + action + observation + answer

The model can follow this pattern when answering the new question.

The examples are **part of the input to this run**.

No training or parameter update takes place.

Source: Yao et al., ReAct (2023).

## Why combine reasoning and acting?

| Approach       | What it provides                             | What can go wrong                            |
|----------------|----------------------------------------------|----------------------------------------------|
| Reasoning only | Intermediate solution steps                  | Invented facts propagate through later steps |
| Act only       | Actions and observations, without “thoughts” | Harder to track and revise a plan            |
| ReAct          | Reasoning with external feedback             | Retrieval or interpretation can still fail   |

**Observations supply evidence. Reasoning organizes its use.**

Source: [Yao et al., ReAct \(2023\), Sections 2–3.](#)

# Who chooses the next step?

## **Workflow: code defines the paths**

Retrieve a record → count slides  
passing QC → report the result.

Known procedure.

Workflows can include branches. Compare flexibility with a simpler baseline.

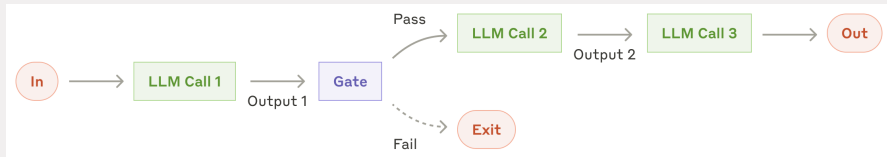
Source: [Building effective agents \(2024\)](#).

## **Agent: the model chooses the next action**

Read the question → choose a tool →  
observe → decide again.

Steps vary with the evidence.

## Prompt chaining: fixed stages



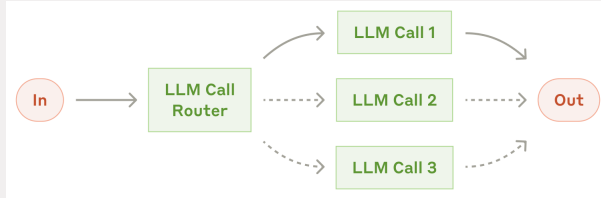
Given image measurements, prepare a report:

**Extract key findings** → **draft a summary** → **translate it.**

Each stage is a predefined LLM call. A check can stop an incomplete draft before translation.

Source: [Building effective agents \(2024\)](#).

## Routing: select a predefined path

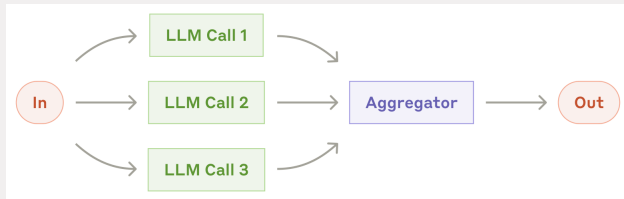


When the question types are known, select a predefined handler:  
**Nucleus count** → **detection**; **tissue area** → **segmentation**.

The router classifies the request. Each handler follows its own defined steps.

Source: [Building effective agents \(2024\)](#).

## Parallelization: independent work runs together



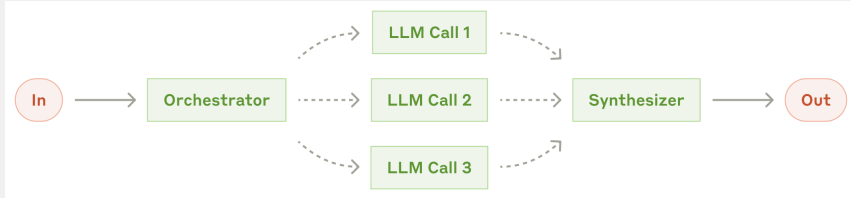
Two independent measurements:

**Segment the image and detect nuclei concurrently, then combine the results.**

The reading uses parallel LLM calls. Here we apply the structure to tools.

Source: [Building effective agents \(2024\)](#).

## Orchestrator-workers: decompose a variable task

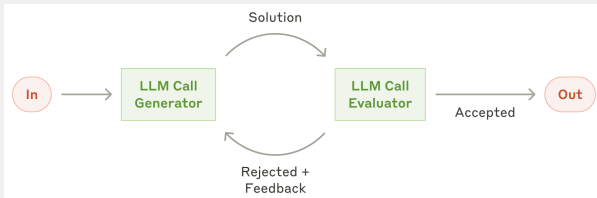


For a broader request, such as “Investigate how these regions differ”:  
**An orchestrator chooses comparisons and assigns them to worker LLMs.**

Workers can use imaging tools. The orchestrator combines their findings.

Source: [Building effective agents \(2024\)](#).

## Evaluator-optimizer: revise against criteria



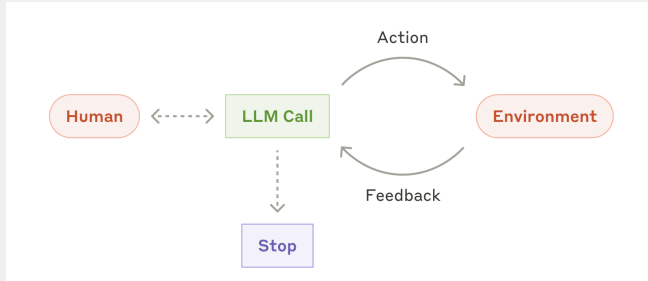
Suppose a draft includes the nucleus count and omits the requested tissue fraction:

**Evaluate the draft → identify the omission → revise.**

Clear criteria make feedback useful: both measurements, correct values and units.

Source: [Building effective agents \(2024\)](#).

# The agent loop in the engineering article



The LLM chooses an action, receives feedback and decides again. It can ask a human for input or stop. The harness enforces the step limit.

Source: [Building effective agents \(2024\)](#).

## Choosing a design for the task

---

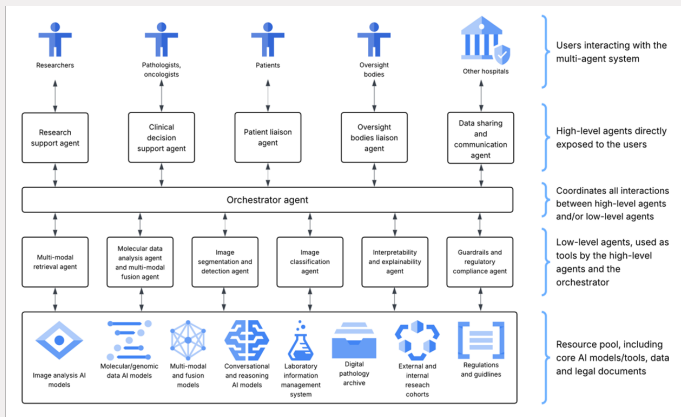
| Task structure                     | A useful starting design       |
|------------------------------------|--------------------------------|
| Always the same two measurements   | Fixed workflow                 |
| Known question types               | Routing to predefined handlers |
| Independent analyses               | Parallel calls                 |
| Variable decomposition             | Orchestrator-workers           |
| Draft improves with clear feedback | Evaluator-optimizer            |
| Model chooses the next action      | Agent loop                     |

---

**My near-term expectation:** fixed workflows will be more practically useful than open-ended agents.

Source: [Pattern definitions: Building effective agents \(2024\)](#).

# A pathology lab agent: a conceptual sketch



# Clinical evaluation needs an interactive task

The third reading is a **commentary proposing AI-SCE:**  
Artificial Intelligence Structured Clinical Examinations.

- Static questions supply the information in advance.
- Simulated encounters test what the agent asks, does and communicates.
- Agent-based modeling can simulate patients and clinicians.

The framework is a conceptual proposal.

Source: [Mehandru et al. \(2024\), commentary, p. 2.](#)

# Evaluate the steps as well as the final answer

The commentary explicitly proposes assessing **intermediate steps**.

- Reasoning: do stated conclusions follow from the evidence?
- Tools and data: were the right inputs and operations used?
- Interactions: did it request information or seek help appropriately?

Two agents can give the same answer through very different processes.  
A trace helps locate errors and decide where checks or human review belong.

Source: [Mehandru et al. \(2024\), commentary, p. 2.](#)

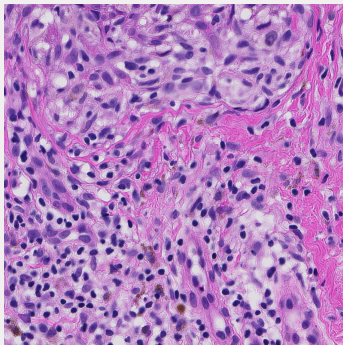
## Building evidence for use in actual care

1. **Simulation:** vary cases and inspect interactions.
2. **External evaluation:** test new settings and populations.
3. **Prospective clinical study:** compare outcomes in the intended care workflow.
4. **Monitoring:** track errors and bias after deployment.

Repeat runs and compare baselines. A successful demo establishes little reliability.

Source: [Mehandru et al. \(2024\)](#), proposed AI-SCE framework.

## The mini-agent: put the pieces together



**Estimate the stroma fraction and the whole-image lymphocyte count.**

Input: an image path.

Outputs: two model-based measurements.

## Two imaging models become callable tools

| Tool           | Operation on image_path                       | Observation returned to the model             |
|----------------|-----------------------------------------------|-----------------------------------------------|
| segment_tissue | Predict a tissue label for each pixel         | Tissue fractions and areas                    |
| detect_nuclei  | Locate nuclei and assign one of three classes | Counts for tumor, lymphocyte and other nuclei |

### Gemma reads tool descriptions and selects requests.

Python runs the imaging models and returns measurements.

# Recognizing a tool call in the model output

**The model generates JSON text:**

```
{"tool": "segment_tissue",  
  "image_path": "image.tif"}
```

- `tool`: the function to run.
- `image_path`: the argument passed to that function.

**The Python harness parses and executes it:**

```
action = json.loads(text)  
observation = functions[action["tool"]](action["image_path"])
```

Illustrative JSON protocol and image path.

## An observation becomes context for the next decision

| Who?  | Recorded sequence (observations abbreviated)                                               |
|-------|--------------------------------------------------------------------------------------------|
| Model | Request <code>segment_tissue(image_path)</code>                                            |
| Tool  | stroma fraction: 0.6994                                                                    |
| Model | Request <code>detect_nuclei(image_path)</code>                                             |
| Tool  | lymphocyte count: 221                                                                      |
| Model | Answer: predicted stroma fraction is 0.699; predicted whole-image lymphocyte count is 221. |

The harness appends each request and observation before the next model call.

# The agent loop, line by line

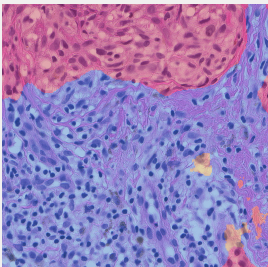
---

| Python-style pseudocode                                                                                                                                                                                                                                                                                                                                | What this line does                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>for step in range(max_steps):<br/>    text = chat(messages)<br/>    action = json.loads(text)<br/>    messages.append(assistant(text))<br/>    if "answer" in action:<br/>        return action["answer"]<br/>    tool = functions[action["tool"]]<br/>    result = tool(action["image_path"])<br/>    messages.append(observation(result))</pre> | <p>Bound the number of turns.</p> <p>Ask the LLM for its next reply.</p> <p>Parse the JSON into fields.</p> <p>Retain the model's reply.</p> <p>Check for a final answer.</p> <p>Return it and stop.</p> <p>Select the requested function.</p> <p>Run it on the image.</p> <p>Add its result for the next turn.</p> |

The limit bounds execution. It does not certify answer quality.

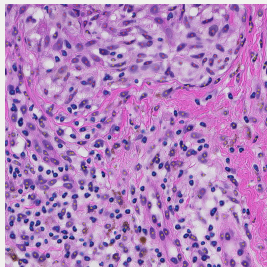
## Two measurements support the answer

### Tissue segmentation



Stroma: **69.9%**

### Nucleus detection



Lymphocyte nuclei: **221**

Recorded predictions. Blue: stroma (left); lymphocyte nuclei (right).

## What the complete system contributes

- **LLM:** chooses requests and composes the answer.
- **Tools:** compute image measurements.
- **Harness:** executes requests, updates context and limits steps.

For this fixed question, two direct tool calls are a useful baseline.

Evaluate measurement accuracy **and** how the agent uses the results.